Introduction to Clean Architecture

Overview



What is Clean Architecture?

Why do we need Clean Architecture?

What does Clean Architecture look like?

What is the responsibility of each layer?

What is Clean Architecture?

- Clean Architecture is a software design philosophy introduced by Robert C. "Uncle Bob" Martin that organizes systems into concentric layers to keep core business logic entirely independent of external frameworks, databases, and user interfaces
- Clean Architecture is a software design approach that helps us organize an application into separate layers, where each layer has a clearly defined responsibility and business logic does not depend directly on databases, frameworks, or the user interface.
- For your teaching demo, let's understand it using your existing Product Web API built with ASP.NET Core, Entity Framework Core, and SQL Server.

Real-World Structure | Maintainable Code | Business Focused

A practical way to build .NET applications that are easier to understand, test, and evolve.

Maintainable Easier to change | **Testable** Business logic in isolation | **Scalable** Supports growth | **Team Friendly** Clear structure | **Future Ready** Swap tech without rewriting business logic

AI = Real Impact

A Real-World Example

A customer should not be able to cancel an order after it has been dispatched.

Where should this rule live?

- In the controller?
- In a service?
- In the repository?
- What about a background job?

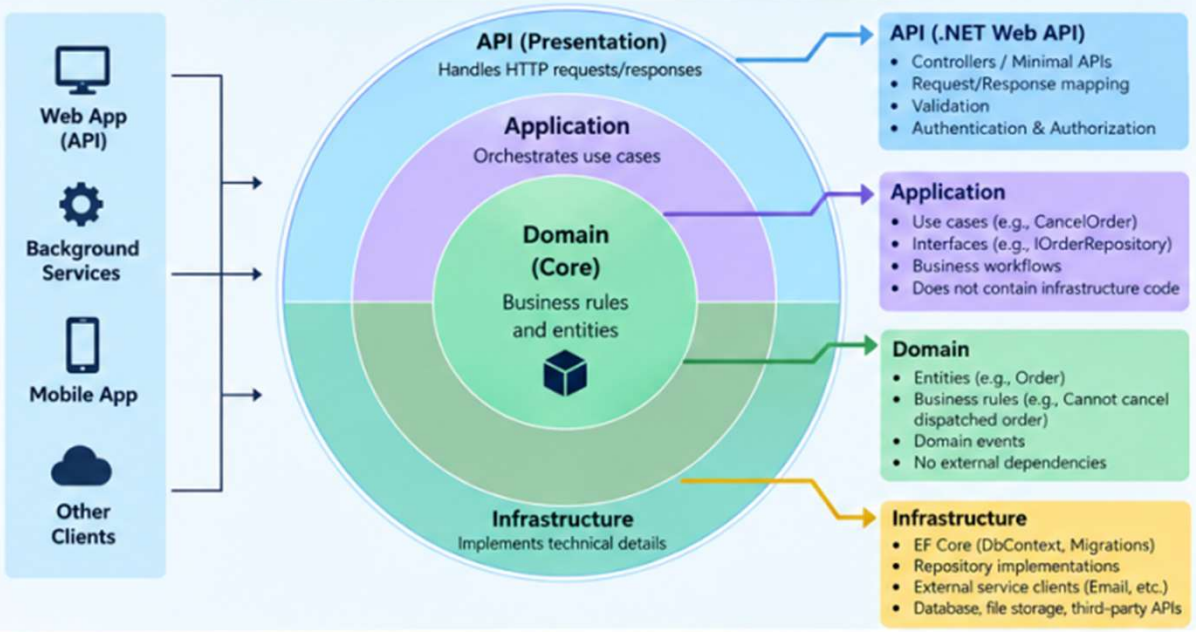
With Clean Architecture, the business rule stays at the center — independent of how it's called.

The Dependency Rule

> **"Source code dependencies point inward."**
>
> – Uncle Bob (Clean Architecture)

Business logic should not know about frameworks, databases, or external services.

Clean Architecture Layers in .NET



Project Structure (Example)

- Solution
  - **Orders.Api**
    - Controllers
    - Program.cs
    - appsettings.json
  - **Orders.Application**
    - UseCases
    - Interfaces
    - DTOs
  - **Orders.Domain**
    - Entities
    - Value Objects
    - Domain Events
  - **Orders.Infrastructure**
    - Data (DbContext)
    - Repositories
    - External Services

1. Domain – Business Rule

```
public class Order
{
    public Guid Id { get; private set; }
    public OrderStatus Status { get; private set; }

    public void Cancel()
    {
        if (Status == OrderStatus.Dispatched)
            throw new InvalidOperationException(
            "A dispatched order cannot be cancelled.");

        Status = OrderStatus.Cancelled;
    }
}
```

2. Application – Use Case

```
public class CancelOrderHandler
{
    private readonly IOrderRepository _repository;

    public async Task Handle(Guid orderId, Guid userId)
    {
        var order = await _repository.GetByIdAsync(orderId);
        if (order == null)
            throw new NotFoundException("Order not found.");

        // Add authorization checks here
        order.Cancel();
        await _repository.SaveAsync(order);
    }
}
```

3. Infrastructure – Repository

```
public class EfOrderRepository : IOrderRepository
{
    private readonly OrdersDbContext _db;

    public EfOrderRepository(OrdersDbContext db)
    {
        _db = db;
    }

    public async Task<Order?> GetByIdAsync(Guid id)
        => await _db.Orders.FindAsync(id);

    public async Task SaveAsync(Order order)
        => await _db.SaveChangesAsync();
}
```

4. API – Endpoint

```
[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    private readonly CancelOrderHandler _handler;

    [HttpPost("{id}/cancel")]
    public async Task<IActionResult> Cancel(Guid id)
    {
        await _handler.Handle(id, GetCurrentUserId());
        return Ok(new { message = "Order cancelled." });
    }
}
```



Mayank Pathak

Why do we need Clean Architecture?

Imagine you have developed a Product Web API with the following features:

- Get all products
- Get a product by ID
- Add a new product
- Update a product
- Delete a product

In a simple Web API, you might write the controller, business logic, and database operations together, or separate them into only a few projects.

Initially, this works well. But as the application grows, problems can arise.

Problem 1: Too much responsibility in one place

Controllers may contain validation, business rules, and database operations. Changes become difficult to manage.

Problem 2: Tight coupling to the database

Business logic that directly depends on Entity Framework Core can be harder to test or reuse with another data-access implementation.

Problem 3: Changes affect multiple parts

Changing how data is stored might require changes to business logic and API code if responsibilities are mixed.

What does Clean Architecture look like?

Clean Architecture — Product Web API

ASP.NET Core Web API · .NET 8 · Entity Framework Core · SQL Server



ProductDemo.API

Controller · DTOs · AutoMapper · Program.cs

- API Controllers (ProductController)
- DTOs (Create, Read, Update)
- AutoMapper configuration
- Program.cs (Dependency Injection)
- Swagger (for testing)





ProductDemo.Application

IProductService interface

- Defines application contracts
- IProductService (CRUD operations)







ProductDemo.Domain

Product entity and validation attributes

- Product entity (same as your implementation)
- Data annotations / validation attributes
- No virtual navigation property (as requested)



ProductDemo.Infrastructure

ECommContext and ProductService implementation

- ECommContext (DbContext with Product DbSet)
- ProductService (implements IProductService)
- EF Core for data access (SQL Server)
- Business logic and CRUD operations (including soft delete)



The Application layer defines the contract; Infrastructure implements it using EF Core.
The API registers the implementation through dependency injection.

What is the responsibility of each layer?

Layer	Responsibility	Example
API	Accept HTTP requests and return responses	ProductController
Application	Define application operations and contracts	IProductService
Domain	Represent core business concepts	Product
Infrastructure	Implement database access and technical details	ProductService, ECommContext